

E-Commerce Order Management System

Event-Driven Microservices · Java 8 · Spring Boot · Spring Cloud · Apache Kafka

Saga choreography with compensation · Complete project documentation

github.com/JavaTechDebasis/ecommerce-microservices

E-Commerce Order Management System (Microservices)

An event-driven **E-Commerce Order Management System** built with **Java 8, Spring Boot, Spring Cloud, and Apache Kafka**. It models a realistic distributed workflow: a customer places an order, stock is reserved, payment is captured, and the customer is notified — coordinated across independent microservices using the **Saga (choreography) pattern** with automatic compensation on failure.

Built as a portfolio / interview project. See [docs/INTERVIEW.md](#) for talking points and likely Q&A.

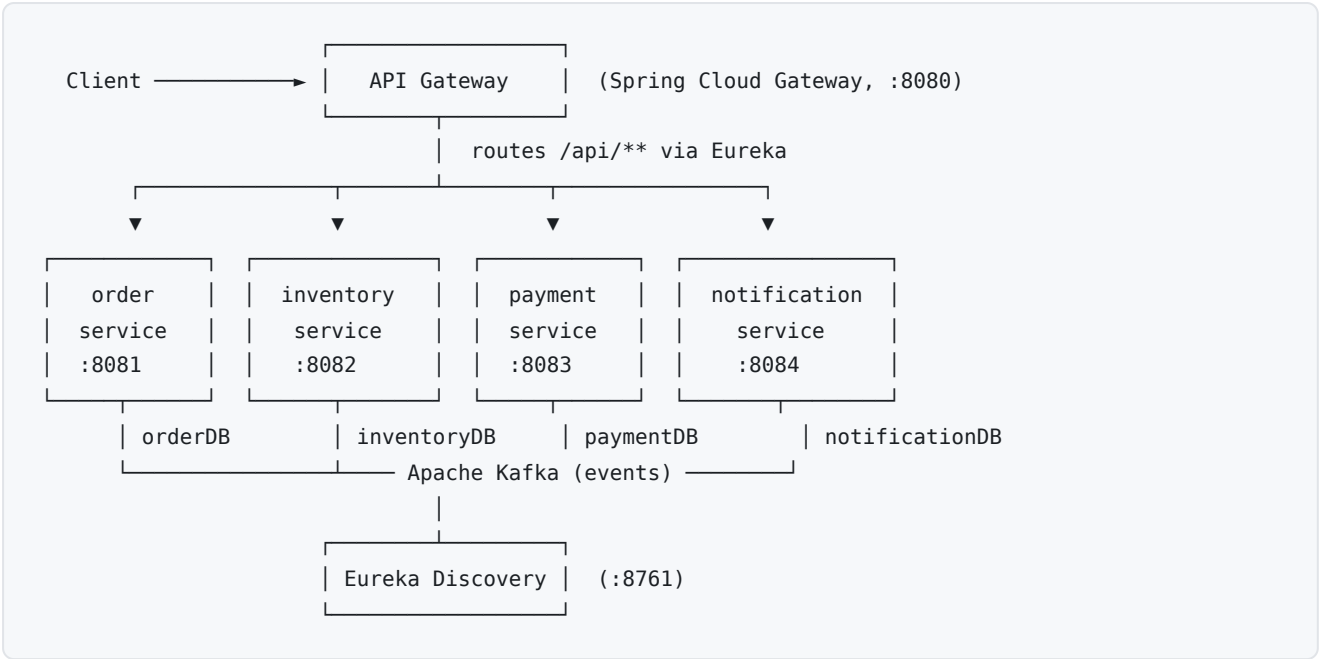
Demo

End-to-end walkthrough of the running system — Eureka registry, then the Saga handling three flows (order confirmed, payment failure with stock compensation, and out-of-stock cancellation):

Tech Stack

Area	Technology
Language	Java 8
Framework	Spring Boot 2.7.x
Microservices	Spring Cloud 2021.0.x (Netflix Eureka, Spring Cloud Gateway, OpenFeign-ready)
Messaging	Apache Kafka (event-driven, Saga choreography)
Persistence	Spring Data JPA + Hibernate; H2 (default) / MySQL (profile)
API	REST (JSON), Bean Validation
Build	Maven (multi-module)
Infra	Docker Compose (Kafka, Zookeeper, Kafka UI, MySQL)
Boilerplate	Lombok

Architecture at a Glance



All services registered with the Eureka discovery server:

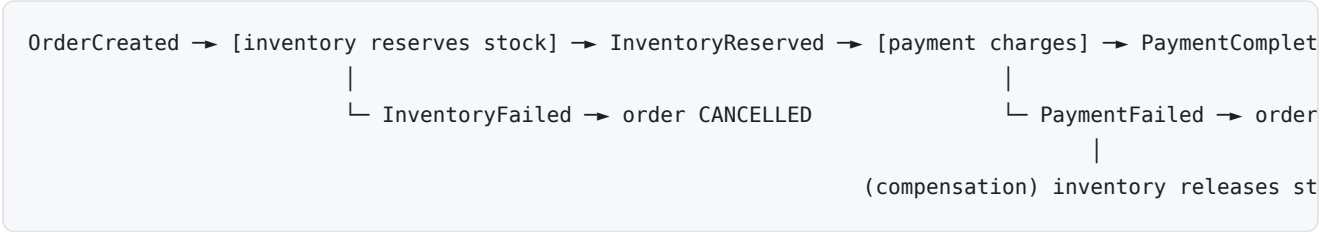
Full details and diagrams:

- [docs/ARCHITECTURE.md](#) — components, design decisions, data ownership
- [docs/SEQUENCE_DIAGRAM.md](#) — Mermaid sequence diagrams for every flow
- [docs/API.md](#) — REST endpoints with request/response examples
- [docs/INTERVIEW.md](#) — how to present this project in interviews

Services

Service	Port	Responsibility
discovery-server	8761	Eureka service registry
api-gateway	8080	Single entry point; routes to services via Eureka
order-service	8081	Owns orders; starts the saga; confirms/cancels orders
inventory-service	8082	Owns stock; reserves/releases inventory
payment-service	8083	Captures payment; emits success/failure
notification-service	8084	Sends customer notifications on terminal events
common-events	—	Shared Kafka event contracts + topic names

The Saga (choreography)



Three flows are demonstrated by `scripts/demo.sh` :

1. **Happy path** → order `CONFIRMED`
 2. **Payment failure** (amount over the approval limit) → order `CANCELLED` + reserved stock released (compensation)
 3. **Out of stock** → order `CANCELLED`
-

Quick Start

Prerequisites

- JDK 8
- Maven 3.6+
- Docker + Docker Compose (for Kafka)

1. Start infrastructure (Kafka + Zookeeper)

```
docker compose up -d zookeeper kafka
# optional UIs / MySQL:
# docker compose up -d kafka-ui mysql
```

2. Build everything

```
mvn -DskipTests clean package
```

3. Start all services

```
./scripts/start-all.sh      # boots all 6 services in order (logs in ./logs)
```

- Eureka dashboard: `http://localhost:8761`
- Kafka UI (if started): `http://localhost:8090`

4. Run the end-to-end demo

```
./scripts/demo.sh
```

5. Stop everything

```
./scripts/stop-all.sh
docker compose down
```

Try a single request

```
curl -X POST http://localhost:8080/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"productCode":"LAPTOP-001","quantity":1,"amount":999,"customerEmail":"alice@example.com"}'

curl http://localhost:8080/api/orders      # watch status go PENDING -> CONFIRMED
```

Seeded products: `LAPTOP-001` (10), `PHONE-001` (25), `HEADSET-001` (3).

Payment approval limit (configurable) is **5000** — orders above it fail to demo compensation.

Switching to MySQL

Each service defaults to in-memory H2 (zero setup). To use MySQL:

```
docker compose up -d mysql
java -jar order-service/target/order-service-1.0.0.jar --spring.profiles.active=mysql
# (repeat for the other JPA services)
```

Repository Layout

```
ecommerce-microservices/
├─ pom.xml                # parent (dependency + module management)
├─ common-events/         # shared Kafka event contracts
├─ discovery-server/      # Eureka
├─ api-gateway/           # Spring Cloud Gateway
├─ order-service/         # REST + JPA + Kafka (producer & consumer)
├─ inventory-service/     # JPA + Kafka (reserve / release)
├─ payment-service/       # JPA + Kafka (charge / refund signal)
├─ notification-service/  # JPA + Kafka (notify)
├─ docker-compose.yml     # Kafka, Zookeeper, Kafka UI, MySQL
├─ scripts/              # start-all / stop-all / demo
└─ docs/                 # ARCHITECTURE, SEQUENCE_DIAGRAM, API, INTERVIEW
```

Author

Debasis Jena — Java Backend Developer

GitHub: [@JavaTechDebasis](#)

Architecture

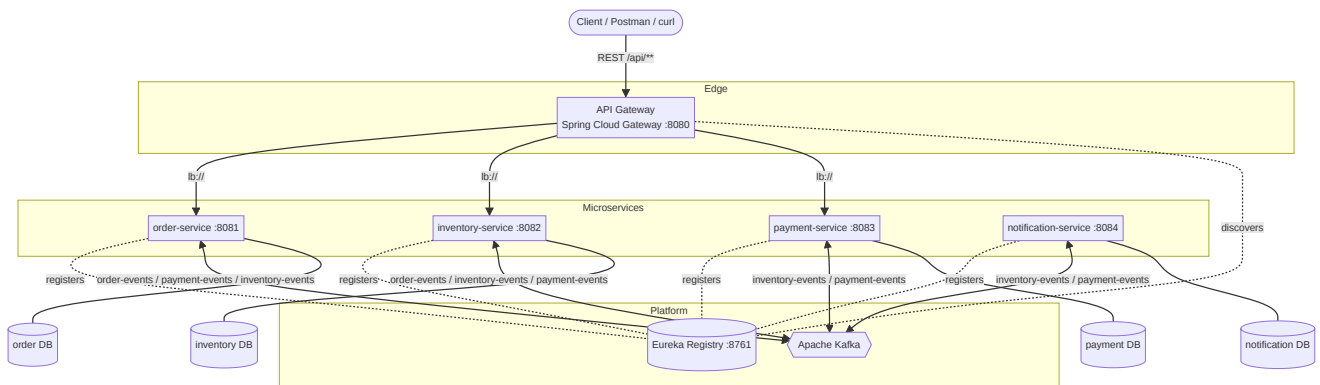
1. Overview

This system implements an **E-Commerce Order Management** workflow as a set of small, independently deployable microservices that communicate **asynchronously over Apache Kafka**. Synchronous client traffic enters through an **API Gateway**; services find each other through a **Eureka** service registry. Each service owns its **own database** (database-per-service), and there are **no synchronous service-to-service calls** in the order workflow — the business process is driven entirely by domain events using the **Saga choreography** pattern.

Design goals

- **Loose coupling** — services interact through events, not direct REST calls, so one service being down doesn't break the others (messages buffer in Kafka).
 - **Data ownership** — every service is the single source of truth for its data.
 - **Resilience** — failures trigger **compensating transactions** instead of distributed locks / 2-phase commit.
 - **Observability** — every state transition is logged; terminal outcomes are persisted as notifications.
-

2. Component View



3. Services and Responsibilities

Service	Owns	Consumes (Kafka)	Produces (Kafka)	REST
order-service	Orders + lifecycle	inventory-events (failed), payment-events (completed/failed)	order-events (OrderCreated)	POST /api/orders , GET /api/orders , GET /api/orders/{id}
inventory-service	Product stock	order-events (created), payment-events (failed → release)	inventory-events (reserved/failed)	GET /api/inventory
payment-service	Payments	inventory-events (reserved)	payment-events (completed/failed)	GET /api/payments
notification-service	Notifications log	inventory-events (failed), payment-events (completed/failed)	—	GET /api/notifications
api-gateway	Routing	—	—	proxies /api/**
discovery-server	Service registry	—	—	Eureka dashboard /

4. Eventing Model

Topics

Topic	Producer	Events carried
order-events	order-service	OrderCreatedEvent
inventory-events	inventory-service	InventoryReservedEvent , InventoryFailedEvent
payment-events	payment-service	PaymentCompletedEvent , PaymentFailedEvent

All event classes live in the shared `common-events` module so producers and consumers share a single contract.

Serialization

- Producers use Spring Kafka's `JsonSerializer` .
- Consumers use `ErrorHandlingDeserializer` wrapping `JsonDeserializer` , with `spring.json.trusted.packages` restricted to the shared event package. The `ErrorHandlingDeserializer` ensures a single malformed ("poison pill") message cannot permanently block a partition.
- The event's fully-qualified type travels in a Kafka header (`__TypeId__`), letting a single listener subscribe to a multi-type topic and dispatch by payload type via class-level `@KafkaListener` + `@KafkaHandler` .

Message keys & ordering

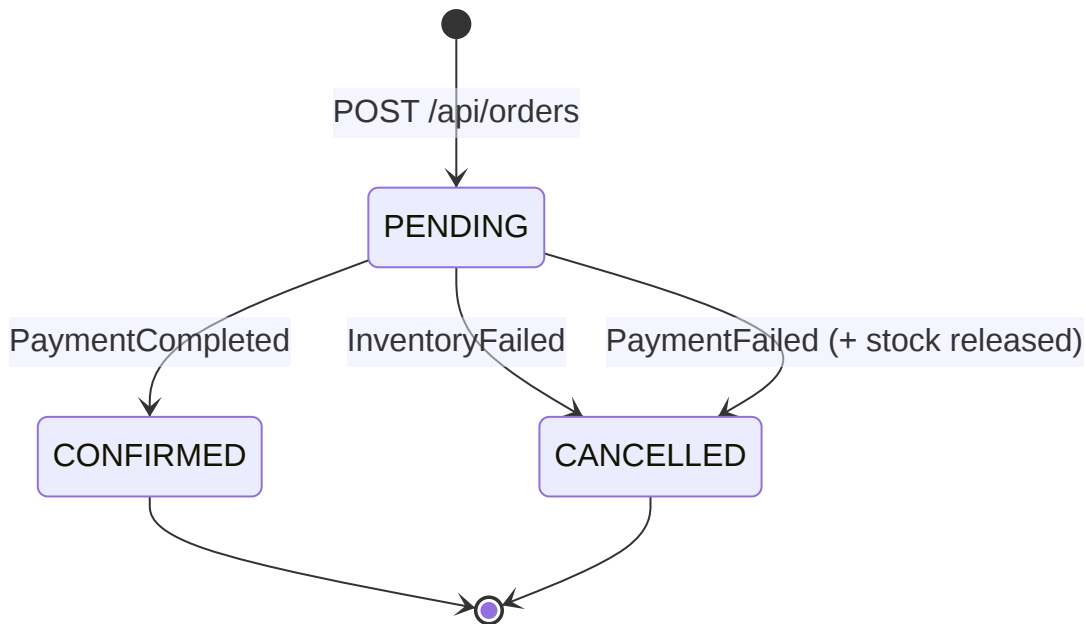
Every event is keyed by `orderId` , so all events for one order land on the same partition and are processed **in order**, which is essential for a correct saga.

5. The Saga (Choreography)

There is no central orchestrator. Each service reacts to events and emits new ones:

1. `order-service` persists the order as **PENDING** and emits `OrderCreatedEvent` .

2. `inventory-service` reserves stock → `InventoryReservedEvent` , or fails → `InventoryFailedEvent` .
3. `payment-service` (on reserved) captures payment → `PaymentCompletedEvent` , or fails → `PaymentFailedEvent` .
4. `order-service` sets the order **CONFIRMED** (payment completed) or **CANCELLED** (inventory/payment failed).
5. **Compensation:** on `PaymentFailedEvent` , `inventory-service` releases the previously reserved stock.
6. `notification-service` reacts to terminal events and notifies the customer.



Why choreography (not orchestration)?

- Fewer moving parts for a workflow of this size; no single point of coordination.
- Services stay autonomous and only depend on event contracts.
- Trade-off: the end-to-end flow is implicit (spread across services). For larger, branchier workflows an **orchestration** saga (e.g. a dedicated orchestrator) is easier to reason about — a great point to raise in interviews.

6. Data Management

- **Database per service** — `order-service` , `inventory-service` , `payment-service` , and `notification-service` each have an isolated schema (H2 by default, MySQL via the `mysql` profile). No shared database, no cross-service joins.
- **Optimistic locking** — `Product` uses a JPA `@Version` column so concurrent reservations of the same product don't lose updates.
- **Idempotency** — `payment-service` checks `existsByOrderId` before charging, so a redelivered `InventoryReservedEvent` does not double-charge.

7. Resilience & Reliability Notes

- **At-least-once delivery:** Kafka consumers may see a message more than once; idempotency keys (`orderId`) guard the critical step.
- **Poison-pill isolation:** `ErrorHandlingDeserializer` prevents deserialization failures from halting consumption.
- **Loose temporal coupling:** if `payment-service` is down, `InventoryReservedEvent` waits in Kafka and is processed when it recovers.
- **Compensation over rollback:** distributed state is reconciled by emitting compensating events rather than holding distributed transactions.

8. Technology Choices

Decision	Rationale
Java 8 + Spring Boot 2.7	Matches a very common enterprise baseline; 2.7 is the last line supporting Java 8.
Kafka	Durable, partitioned, replayable log — ideal for event-driven sagas.
Eureka + Spring Cloud Gateway	Battle-tested Netflix/Spring Cloud stack for discovery + edge routing.
Database-per-service	Enforces service autonomy and clear ownership.
H2 default, MySQL profile	Runs anywhere with zero setup, but production-like DB is one flag away.
Multi-module Maven	Single build, shared event contracts, consistent dependency management.

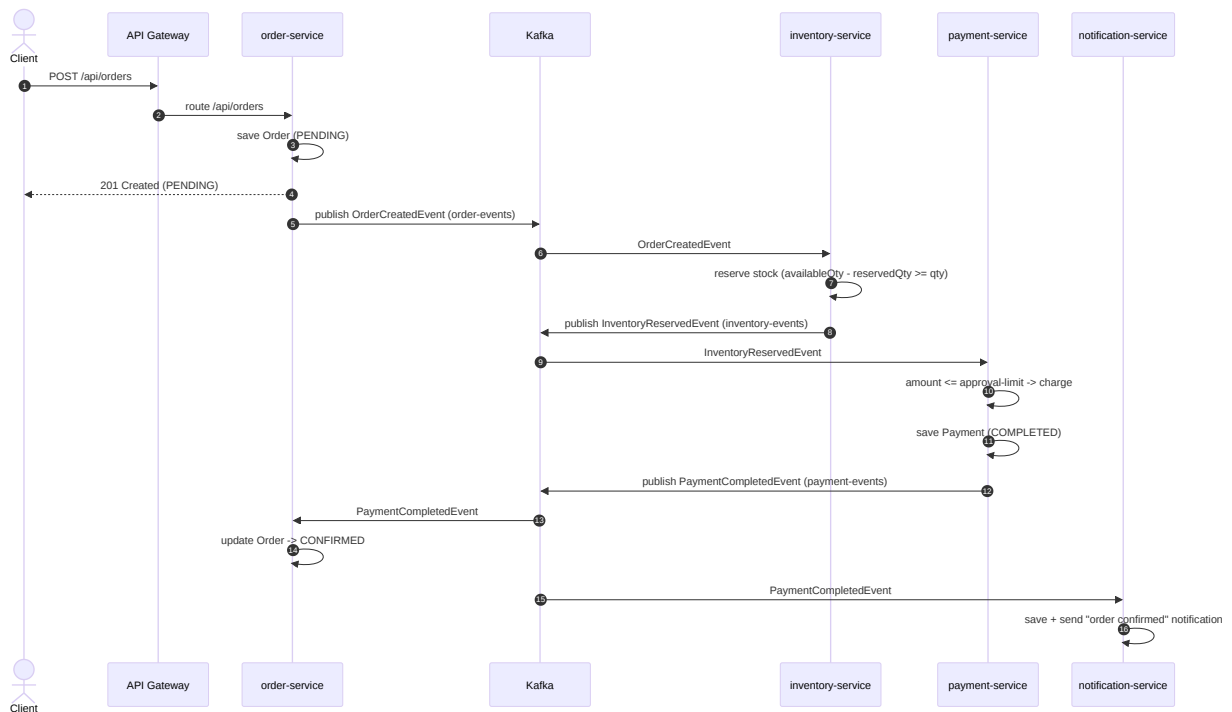
9. Possible Extensions (talking points)

- Replace H2 with MySQL/PostgreSQL per service in production.
- Add **Spring Cloud Config** for centralized configuration.
- Add **Resilience4j** (circuit breaker / retry) around any synchronous calls.
- Add **distributed tracing** (Micrometer Tracing + Zipkin) to follow a saga across services.
- Add the **transactional outbox** pattern to make "save to DB + publish event" atomic.
- Add a **dead-letter topic** for messages that fail repeatedly.

Sequence Diagrams

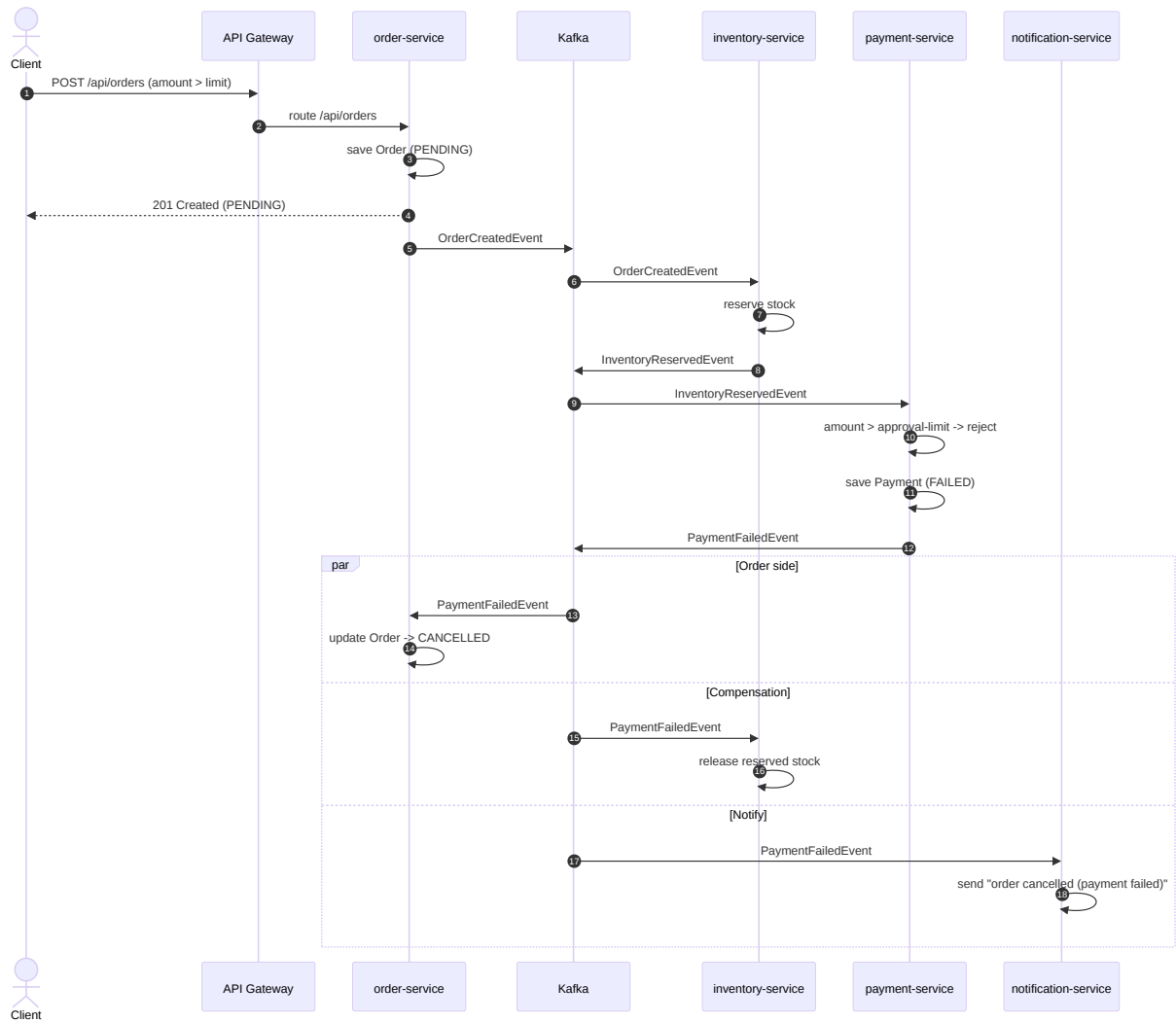
These diagrams use [Mermaid](#), which renders automatically on GitHub. Each diagram shows one flow through the choreography saga.

1. Happy Path — Order Confirmed



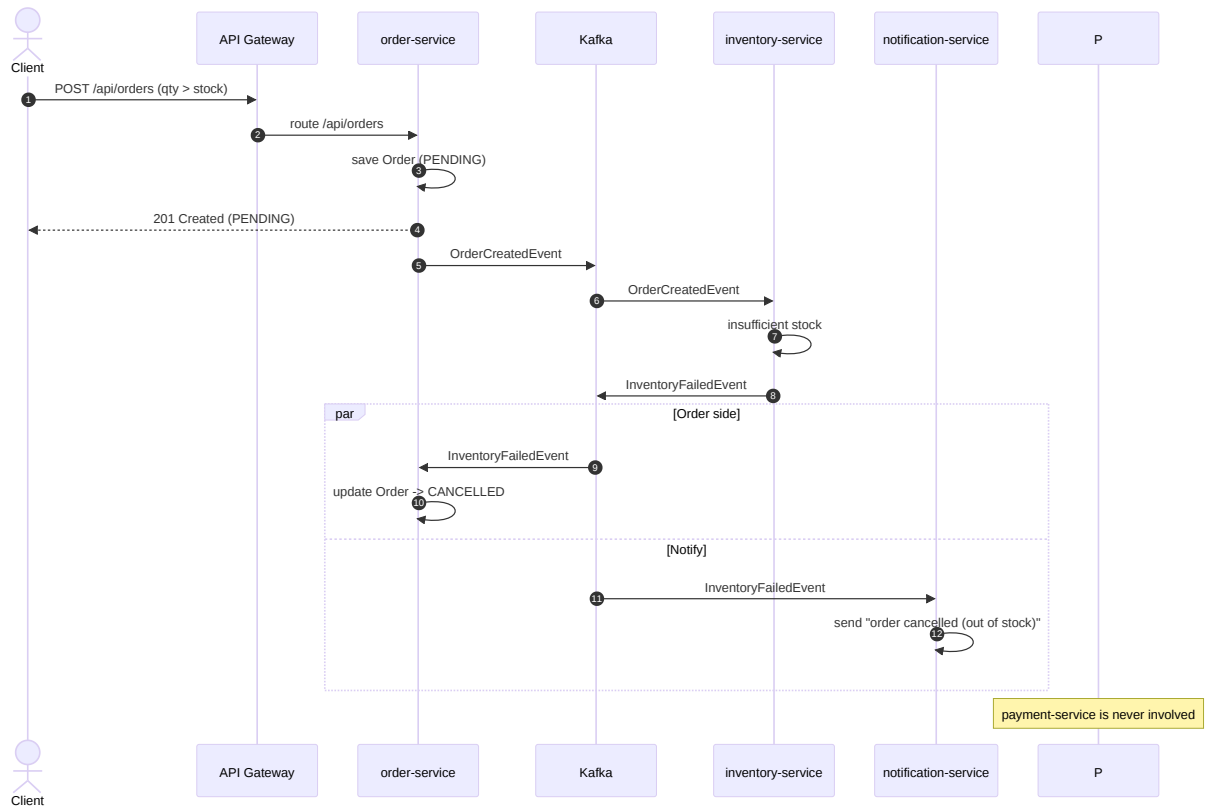
2. Payment Failure — Order Cancelled + Compensation

The amount exceeds the payment approval limit, so payment fails. The order is cancelled **and** the inventory previously reserved is released (the compensating action).

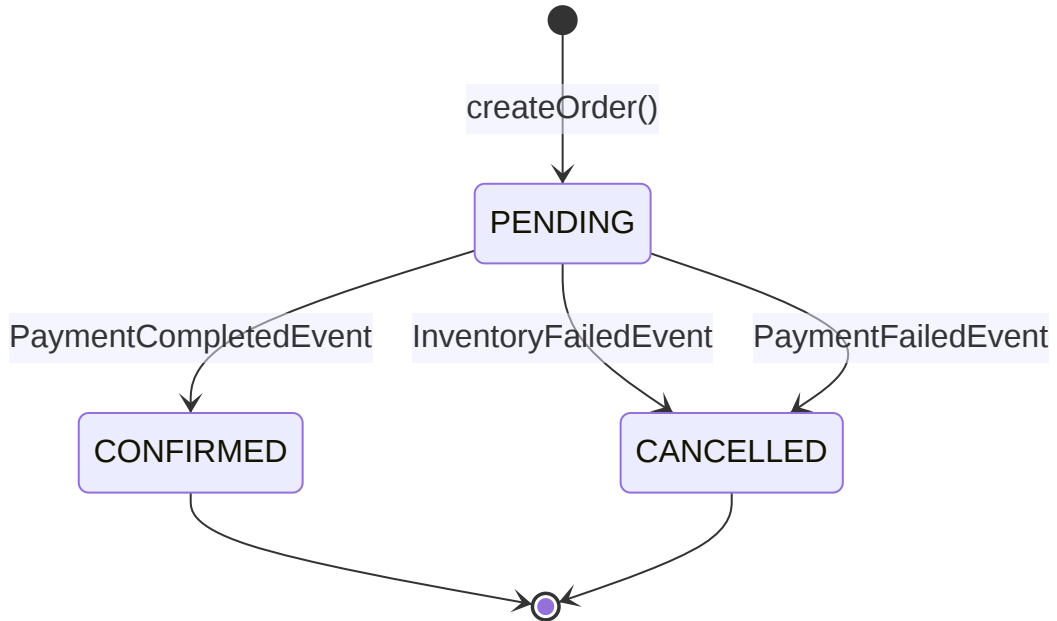


3. Out of Stock — Order Cancelled Early

Inventory reservation fails immediately, so the saga short-circuits before payment.



4. State Machine (Order)



API Reference

All client traffic goes through the **API Gateway** at `http://localhost:8080` .
Individual services can also be hit directly on their own ports (useful for debugging).

Resource	Via Gateway	Direct
Orders	<code>http://localhost:8080/api/orders</code>	<code>http://localhost:8081/api/orders</code>
Inventory	<code>http://localhost:8080/api/inventory</code>	<code>http://localhost:8082/api/inventory</code>
Payments	<code>http://localhost:8080/api/payments</code>	<code>http://localhost:8083/api/payments</code>
Notifications	—	<code>http://localhost:8084/api/notifications</code>

Order Service

Create an order

`POST /api/orders`

Request body:

```
{
  "productCode": "LAPTOP-001",
  "quantity": 1,
  "amount": 999,
  "customerEmail": "alice@example.com"
}
```

Field	Type	Rules
<code>productCode</code>	string	required, not blank
<code>quantity</code>	int	required, >= 1
<code>amount</code>	number	required, > 0
<code>customerEmail</code>	string	required, valid email

Response `201 Created` (order starts in `PENDING` ; the saga then moves it to `CONFIRMED` / `CANCELLED` asynchronously):

```
{
  "id": 1,
  "productCode": "LAPTOP-001",
  "quantity": 1,
  "amount": 999.00,
  "customerEmail": "alice@example.com",
  "status": "PENDING",
  "statusReason": null,
  "createdAt": "2026-06-21T14:35:43.521Z",
  "updatedAt": "2026-06-21T14:35:43.521Z"
}
```

Validation error `400 Bad Request` :

```
{
  "timestamp": "2026-06-21T14:40:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "errors": { "customerEmail": "customerEmail must be a valid email" }
}
```

Get an order

`GET /api/orders/{id}` → `200 OK` (or `404 Not Found`)

List orders

`GET /api/orders` → `200 OK` (array)

`status` is one of `PENDING` , `CONFIRMED` , `CANCELLED` ; `statusReason` explains a cancellation.

Inventory Service

List products / current stock

`GET /api/inventory` → `200 OK`

```
[
  {
    "id": 1, "productCode": "LAPTOP-001", "name": "14-inch Laptop",
    "price": 999.00, "availableQuantity": 10, "reservedQuantity": 1, "version": 3
  }
]
```

Seeded catalogue: `LAPTOP-001` (10), `PHONE-001` (25), `HEADSET-001` (3).

Payment Service

List payments

GET /api/payments → 200 OK

```
[
  {
    "id": 1, "orderId": 1,
    "paymentReference": "PAY-59677b39-...", "amount": 999.00,
    "customerEmail": "alice@example.com", "status": "COMPLETED", "failureReason": null,
    "createdAt": "2026-06-21T14:35:44Z"
  }
]
```

status is COMPLETED or FAILED . Orders above the configured **approval limit** (payment.approval-limit , default 5000) fail.

Notification Service

List notifications

GET /api/notifications → 200 OK

```
[
  {
    "id": 2, "orderId": 1, "recipient": "alice@example.com", "channel": "EMAIL",
    "message": "Your order #1 is confirmed. Payment PAY-... of 999 was successful.",
    "sentAt": "2026-06-21T14:35:44Z"
  }
]
```

Actuator

Every service exposes GET /actuator/health (and /info). The gateway also exposes /actuator/gateway .

Example: full happy-path with curl

```
# place an order
curl -X POST http://localhost:8080/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"productCode":"LAPTOP-001","quantity":1,"amount":999,"customerEmail":"alice@example.com"}'

# a moment later, the order is CONFIRMED
curl http://localhost:8080/api/orders
curl http://localhost:8080/api/payments
curl http://localhost:8084/api/notifications
```

Interview Guide

This file helps you **present this project confidently** and answer the questions interviewers commonly ask about microservices, Kafka, and Spring Boot. Read the [ARCHITECTURE.md](#) and [SEQUENCE_DIAGRAM.md](#) alongside this.

1. 30-Second Pitch

"I built an **event-driven E-Commerce Order Management System** using Java 8, Spring Boot, Spring Cloud, and Apache Kafka. It's split into independent microservices — order, inventory, payment, and notification — behind an API Gateway with Eureka service discovery. The order workflow is implemented as a **Saga using Kafka choreography**: when an order is placed, inventory is reserved, payment is captured, and the customer is notified, all through asynchronous events. If payment fails, a **compensating transaction** automatically releases the reserved stock and cancels the order. Each service has its own database, so they're fully decoupled."

2. Why each technology (be ready to justify)

Tech	Why I used it
Microservices	Independent deployment, scaling, and failure isolation; each team/service owns a bounded context.
Kafka	Durable, ordered, replayable event log; decouples services in time (a down service just catches up later).
Saga pattern	Maintains data consistency across services without distributed transactions / 2-phase commit.
Eureka	Client-side service discovery so services find each other by name, not hard-coded URLs.
API Gateway	Single entry point: routing, and a natural place for cross-cutting concerns (auth, rate limiting, logging).
Database per service	Enforces autonomy; no shared schema means services can evolve independently.
H2 / MySQL profile	Runs anywhere instantly, but production DB is one flag away.

3. The Saga, explained simply

There are two ways to coordinate a multi-service transaction:

- **Orchestration** — a central "orchestrator" tells each service what to do.
- **Choreography** — each service listens for events and reacts, emitting new events. *(This project uses choreography.)*

Flow: OrderCreated → InventoryReserved → PaymentCompleted → Order CONFIRMED .

On failure, **compensating events** undo prior steps: e.g. PaymentFailed → inventory releases stock → Order CANCELLED .

Why choreography here? The workflow is short and linear, so a central orchestrator would be overkill. I can clearly articulate the trade-off (see Q&A below), which is what interviewers want.

4. Likely Questions & Strong Answers

Q: What is a Saga and why not just use a distributed transaction (2PC)?

A: A Saga breaks one distributed transaction into a sequence of **local** transactions, each publishing an event that triggers the next. If a step fails, earlier steps are undone with **compensating transactions**. 2PC needs a coordinator and distributed locks that hurt availability and don't scale well; sagas keep services autonomous and highly available at the cost of only **eventual** consistency.

Q: Choreography vs orchestration — which did you use and why?

A: Choreography — services react to events with no central coordinator. It's simpler for a short, linear flow and keeps services decoupled. The downside is the end-to-end flow is implicit (spread across services), so for large, branching workflows I'd switch to orchestration for clarity and easier monitoring.

Q: How do you guarantee ordering of events?

A: I key every event by `orderId`. Kafka guarantees ordering **within a partition**, and a given key always maps to the same partition, so all events for one order are processed in order.

Q: Kafka delivers at-least-once — how do you handle duplicates?

A: Idempotency. For example, `payment-service` checks `existsByOrderId` before charging, so a redelivered `InventoryReservedEvent` won't double-charge. Compensations are written to be safe to apply once.

Q: What's a "poison pill" and how did you handle it?

A: A message that always fails to deserialize/process and blocks the partition. I use `ErrorHandlingDeserializer` wrapping `JsonDeserializer`, so a bad message is logged/skipped instead of stalling the consumer. In production I'd route it to a **dead-letter topic**.

Q: How do producer and consumer agree on the event format?

A: A shared `common-events` module holds the event classes and topic names, so both sides compile against the same contract. The concrete type travels in the `__TypeId__` Kafka header, letting one listener handle multiple event types via `@KafkaHandler`.

Q: "Save to DB and publish to Kafka" isn't atomic. What if the app crashes between them?

A: Correct — that's the dual-write problem. The robust fix is the **Transactional Outbox** pattern: write the event to an `outbox` table in the same DB transaction, then a relay (e.g. Debezium CDC) publishes it to Kafka. I called this out as a planned extension.

Q: How does service discovery work?

A: Each service registers with **Eureka** on startup. The gateway and clients resolve a logical name (e.g. `lb://order-service`) to a live instance via Eureka, with client-side load balancing — no hard-coded hosts/ports.

Q: How would you scale this?

A: Run multiple instances of a service in the same Kafka **consumer group** — Kafka distributes partitions across instances so they share the load while preserving per-key ordering. Stateless services + database-per-service make horizontal scaling straightforward.

Q: How do you handle a service being down?

A: Because communication is asynchronous, events simply accumulate in Kafka and are processed when the service recovers (temporal decoupling). For any synchronous calls I'd add **Resilience4j** circuit breakers/retries.

Q: How would you debug a request that spans services?

A: Add **distributed tracing** (Micrometer Tracing + Zipkin/Jaeger) with a correlation/trace id propagated through Kafka headers, so I can see the full saga timeline.

Q: Why Java 8 / Spring Boot 2.7?

A: It mirrors a very common enterprise baseline. Spring Boot 2.7 is the last line that supports Java 8. I can also discuss migrating to Java 17 + Boot 3.x (Jakarta namespace change).

5. What to Demo Live

1. `docker compose up -d zookeeper kafka` then `./scripts/start-all.sh`.
 2. Show the **Eureka dashboard** (<http://localhost:8761>) — all services registered.
 3. Run `./scripts/demo.sh` and walk through the three flows:
 - Happy path → **CONFIRMED**
 - Payment over limit → **CANCELLED** + stock released (compensation)
 - Out of stock → **CANCELLED**
 4. Open **Kafka UI** (<http://localhost:8090>) and show the messages on `order-events`, `inventory-events`, `payment-events`.
 5. Point at the **logs** to show events flowing between services.
-

6. Honest Talking Points (shows maturity)

- Choreography is implicit; for bigger flows I'd add an orchestrator or a saga framework.
 - The DB+Kafka write isn't atomic yet → outbox pattern is the fix.
 - No auth on the gateway yet → would add JWT/OAuth2 at the edge.
 - H2 in-memory for demos; production needs real per-service databases, schema migrations (Flyway/Liquibase), and a dead-letter strategy.
-

7. Resume Bullet Points (copy/adapt)

- Designed and built an **event-driven microservices** e-commerce order system (Java 8, Spring Boot, Spring Cloud, Apache Kafka) using the **Saga choreography** pattern with automatic **compensating transactions**.
- Implemented **database-per-service** with Spring Data JPA and asynchronous, ordered event processing keyed by order id for correctness under concurrency.
- Added **service discovery (Eureka)** and an **API Gateway (Spring Cloud Gateway)** as the single entry point for all client traffic.
- Hardened Kafka consumers with **idempotency**, **optimistic locking**, and **poison-pill handling** (`ErrorHandlingDeserializer`).
- Containerized infrastructure with **Docker Compose** (Kafka, Zookeeper, MySQL) and automated end-to-end demo scripts.